🎓 IntelliStudy: AI Learning & Study Assistant

Complete Team Work Distribution, Soft Computing Mapping & Viva Defense Guide

Course: Soft Computing (CS/MCA) **Architecture:** Zero-LangChain Semantic RAG **Team Size:** 4 Members
Models: Gemini 3.5 Flash + all-MiniLM-L6-v2

👥 1. Team Work Division & Module Ownership

The codebase is modularized so that each team member owns complete end-to-end features, individual files, and theoretical components that can be independently defended during the project viva.

1. Yusra (Team Lead & Pipeline Architect)

LEAD & SEMANTIC SEARCH

Core Files Owned: `rag.py` (Extraction, Chunking & FAISS) • `config.py` (Environment & Network Proxy Resilience)

- **PDF Ingestion & Validation:** Implemented page-by-page extraction using `pypdf` with automatic detection and graceful rejection of image-only/scanned PDFs.
- **Context-Aware Text Chunking:** Designed a 700-character sliding chunk window with a 100-character overlap, snapping cuts to natural sentence boundaries so context isn't severed.
- **Dense Embedding Pipeline:** Integrated `sentence-transformers/all-MiniLM-L6-v2` to convert passages into 384-dimensional dense semantic vectors.
- **FAISS Vector Database:** Built, loaded, and saved the vector index using `faiss.IndexFlatIP` on L2-normalized vectors to achieve maximum-speed cosine similarity retrieval.
- **Network & Offline Resilience:** Engineered proxy bypass and local cache loading (`local_files_only=True`) to eliminate institutional proxy 407 timeouts.

30-Second Viva Pitch: *"I led the project and built the knowledge ingestion and semantic vector retrieval pipeline. I took raw student PDF notes, chunked them with sliding boundary preservation, projected them into a 384-dimensional semantic embedding space using a dense neural model, and indexed them locally with FAISS so queries match by meaning rather than simple keywords."*

2. Sahil Tamang (Generative AI & QA Engineer)

RAG QA & ANTI-HALLUCINATION

Core Files Owned: `rag.py` (Answer Generation) • `tools.py` (Intent Routing)

- **Strict Grounded Prompt Engineering:** Crafted rigorous system prompt templates for Google Gemini (`gemini-3.5-flash`) mandating answers strictly based on retrieved context.
- **Anti-Hallucination Confidence Guardrails:** Set an approximate boundary cutoff score of `0.18`. If retrieval confidence falls below 0.18, the model politely refuses rather than fabricating false facts.
- **Transparent Source Attribution:** Engineered the citation module linking every generated answer to exact file names, page numbers, similarity match scores, and text snippets.
- **Query Intent Router:** Built the deterministic keyword classifier in `tools.py` that routes user queries between Q&A, quiz creation, study plans, or memory status without wasting LLM tokens.

30-Second Viva Pitch: *"I designed the grounded RAG answer generation module. I ensured Gemini remains strictly grounded on the retrieved PDF passages, established a 0.18 cosine similarity confidence threshold to eliminate hallucinations, and built the citation engine that gives students exact document and page references for every answer."*

3. Mohd Sufiyan (Adaptive Quiz Engine & Analytics Specialist)

QUIZ ENGINE & MEMORY

Core Files Owned: `quiz.py` (MCQ Generation & Grading) • `memory.py` (Student Profile & Analytics)

- **Structured MCQ Generator:** Prompted Gemini to output strict JSON arrays of 5 or 10 questions with options A–D, correct answer key, conceptual explanations, and sub-topics.
- **Robust JSON Cleaner & Backup Bank:** Built regex extractors to strip Markdown fences and created static backup question banks so quizzes function even offline or under API quota limits.
- **Deterministic Objective Grading:** Engineered pure rule-based marking (matching user selection to stored letter keys) ensuring zero AI hallucinations or marking bias.
- **Student Memory Persistence:** Created `data/memory.json` to log inquiries, attempts, average scores, and track individual learning progression over time.
- **Mastery & Weak Topic Heuristics:** Formulated adaptive scoring thresholds: scores < 70% flag topics as priority weak areas, while scores ≥ 85% promote topics to Mastered.

30-Second Viva Pitch: *"I developed the quiz generation and student analytics system. I built structured JSON parsing with fallback question banks for offline reliability, created deterministic auto-marking for fairness, and implemented the heuristic algorithms that categorize a student's weak versus mastered areas based on performance."*

4. Abhishek Kapoor (UI/UX System & Study Planner)

STREAMLIT UI & STUDY PLANNER

Core Files Owned: `app.py` (Application Shell) • `ui_components.py` (Modern SaaS Design System) • `study_plan.py`

- **Multi-Page Streamlit Architecture:** Designed and implemented the complete 6-page navigation flow (Dashboard, Study Materials, Ask AI, Quiz, My Progress, Study Plan).
- **Custom SaaS CSS Design System:** Created `ui_components.py` featuring Plus Jakarta Sans typography, card containers, badge components, metric widgets, and animated steppers.
- **State Management:** Managed `st.session_state` across multiple page switches, preserving active quizzes, chat histories, and uploaded file states.
- **Dynamic 4-Phase Study Planner:** Built the revision algorithm that parses exam date, daily available study hours, knowledge level, and weak topics into an actionable timetable: (Phase 1: Theory, Phase 2: Active Recall, Phase 3: Weak Area Drilling, Phase 4: Final Revision).
- **Interactive Checklist & Export:** Created interactive completion tracking with checkboxes and one-click Markdown (`.md`) schedule file export.

30-Second Viva Pitch: *"I built the interactive Streamlit user interface and the dynamic study planner. I created a custom modern SaaS design system with CSS components, handled multi-page state persistence, and programmed the scheduling algorithm that converts exam deadlines and identified weak topics into a phased daily study plan."*

Summary Matrix of Responsibilities

Member	Primary Module	Main Technologies	Core Contribution
Yusra	`rag.py`, `config.py`	PyPDF, SentenceTransformers, FAISS	PDF text extraction, chunking, 384-d vector embeddings, FAISS indexing & proxy bypass
Sahil Tamang	`rag.py`, `tools.py`	Gemini 3.5 Flash, RAG Architecture	Anti-hallucination prompt engineering, 0.18 confidence threshold & source citation
Mohd Sufiyan	`quiz.py`, `memory.py`	JSON Parsing, Heuristics, File I/O	Structured MCQ generation, offline fallback bank, deterministic grading & memory tracking
Abhishek Kapoor	`app.py`, `ui_components.py`, `study_plan.py`	Streamlit, Custom CSS, Datetime Math	6-page UI/UX system, CSS SaaS cards, session state management & phased study schedule engine

🧠 2. Soft Computing Concepts Applied in IntelliStudy

Soft Computing differs from conventional (hard) computing by tolerating **imprecision, uncertainty, partial truth, and approximation** to achieve tractability, robustness, and low solution cost. Here is how your project embodies Soft Computing principles:

A. Neural Distributed Representations (Dense Vector Embeddings)

Hard Computing: Keyword search (e.g., regex, SQL `LIKE`) requires exact string matches. If a user queries "*CPU scheduling*" and the PDF writes "*Processor task dispatching*", hard search returns zero results.

Soft Computing (Neural Latent Space): Using `sentence-transformers/all-MiniLM-L6-v2` (a 6-layer MiniLM neural network), each text chunk is mapped into a continuous **384-dimensional latent semantic space**. Semantically related concepts cluster geometrically together regardless of differing vocabulary.

B. Vector Space Geometry & Metric Learning (Cosine Similarity via FAISS)

Soft computing replaces boolean matching (True/False) with **degrees of similarity** in high-dimensional space.

$$\text{Cosine Similarity}(\mathbf{u}, \mathbf{v}) = \cos(\theta) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\|_2 \|\mathbf{v}\|_2}$$

By L2-normalizing vectors ($\|\mathbf{u}\|_2 = 1, \|\mathbf{v}\|_2 = 1$), cosine similarity simplifies directly to the **Inner Product** ($\mathbf{u} \cdot \mathbf{v}$). This allows `faiss.IndexFlatIP` to execute millions of vector comparisons in microseconds using CPU vector instructions.

C. Uncertainty Handling & Approximate Reasoning (Similarity Thresholding)

A major flaw in naive generative AI is hallucination—confidently inventing false information when uncertain.

Soft Computing Solution: We implement an approximate decision boundary at **$S_{\text{threshold}} = 0.18$**:

- $\text{Similarity Score} \ge 0.18 \implies$ High contextual confidence; proceed with conditioned LLM inference.
- $\text{Similarity Score} < 0.18 \implies$ Uncertain / irrelevant context; suppress generation and return a polite fallback refusal.

D. Hybrid Computing: Probabilistic Reasoning + Deterministic Rules

Soft computing systems often combine neural components with deterministic heuristics to ensure reliability:

- **Probabilistic Soft Computing:** Gemini generates natural language answers and novel quiz questions by modeling conditional token probability distributions $P(w_t \mid w_{<t}, \text{context})$.
- **Deterministic Crisp Computing:** Quiz marking and student memory score updates bypass AI entirely. Comparing student responses against stored answer keys guarantees 100% fair, reproducible, and verifiable evaluations.

E. Adaptive Learning & Dynamic Student Profiling

Inspired by fuzzy membership functions, student topic mastery is continuously updated based on performance metrics:

- $\text{Score} < 70\% \implies$ High membership in **Weak Topics** (triggers priority allocation in Study Plan).
- $70\% \le \text{Score} < 85\% \implies$ Intermediate progress (retained for general practice).
- $\text{Score} \ge 85\% \implies$ Promoted to **Mastered Topics** (cleared from weak queue).

🎯 3. High-Yield Viva Questions & Answers by Member

A. Questions for Yusra (Team Lead & Semantic Search)

Q1. Why did you choose FAISS IndexFlatIP instead of IndexL2 (Euclidean distance)?

Answer: Euclidean distance (L2) measures the absolute geometric distance between vectors, which can be distorted by passage length. Cosine similarity measures the angle between vectors, reflecting purely semantic orientation. Because we normalize all embedding vectors to unit length using `faiss.normalize_L2`, the Inner Product (IP) is mathematically identical to Cosine Similarity, which runs significantly faster on CPU.

Q2. What is the significance of the 100-character chunk overlap?

Answer: If a key definition or concept falls directly on the boundary of chunk N and chunk $N+1$, splitting without overlap cuts the sentence in half, destroying its semantic meaning. The 100-character sliding overlap guarantees that boundary sentences remain intact in at least one chunk.

Q3. What happened when the app gave a 407 error, and how did you resolve it?

Answer: The Windows environment had an unauthenticated campus proxy (`172.31.100.29:3128`) enabled in Internet Settings. Python libraries attempted to route Hugging Face and Gemini requests through this proxy, which rejected them with 407 Proxy Authentication Required. I resolved it by configuring proxy environment bypasses in `config.py`, enabling REST transport in Gemini, and setting `local_files_only=True` for SentenceTransformers.

B. Questions for Sahil Tamang (Generative AI & QA)

Q1. What is RAG, and why is it superior to fine-tuning an LLM on student notes?

Answer: Retrieval-Augmented Generation (RAG) retrieves relevant external documents at query time and passes them as context to the LLM. Fine-tuning is expensive, slow, prone to hallucinations, cannot cite exact page numbers, and requires retraining whenever a new PDF is added. RAG allows instant PDF updates with zero training cost and exact page citations.

Q2. How did you prevent Gemini from hallucinating answers?

Answer: Through two layers: (1) System prompt constraints explicitly instructing the model: *"If the answer cannot be found in the provided context, state: 'I couldn't find this information in the uploaded study material.'"* (2) Similarity score thresholding: if the top FAISS retrieved chunk has a score below 0.18, Gemini is not even called, and the refusal is returned immediately.

Q3. Why use Gemini 3.5 Flash instead of larger models like Gemini Pro or GPT-4?

Answer: `gemini-3.5-flash` is optimized for sub-second latency, low memory footprint, and high reasoning throughput. For a RAG application where the relevant knowledge is already provided in the prompt context, Flash provides near-instant grounded answers without the latency overhead of heavier models.

C. Questions for Mohd Sufiyan (Quiz Engine & Analytics)

Q1. Why didn't you use Gemini to mark the student's quiz answers?

Answer: LLMs are probabilistic and non-deterministic; they can output different grades for the same answer, hallucinate, or fail if internet drops. By comparing the student's selected letter (e.g. 'B') directly against the stored correct answer key in pure Python, evaluation is 100% deterministic, immediate, free of token costs, and completely fair.

Q2. How do you handle cases where the LLM returns invalid JSON for a quiz?

Answer: I implemented regex sanitization to strip markdown fences (```` ```json ````), followed by strict dictionary validation. If the JSON is still malformed or the API is unavailable, the system automatically falls back to an offline curated question bank so the student never experiences a crash.

Q3. How does the student memory system classify mastery?

Answer: In `memory.py`, quiz scores under 70% automatically add the topic and its tested sub-topics into `weak_topics`. When the student retakes a quiz and scores 85% or higher, the topic is marked as mastered and removed from the weak topic queue.

D. Questions for Abhishek Kapoor (UI/UX System & Study Planner)

Q1. How does Streamlit handle state when a user switches between the 6 pages?

Answer: Streamlit reruns the entire script upon every user interaction. To prevent data loss, I utilized `st.session_state` to persist variables such as `current_page`, `chat_history`, active quiz answers, and uploaded file metadata across reruns.

Q2. How does the Study Plan Generator construct its 4 phases?

Answer: It calculates days remaining using `datetime.strptime(exam_date) - date.today()`. It then allocates available days into 4 logical phases: Phase 1 (Core Theory), Phase 2 (Active Recall & Practice), Phase 3 (Drilling Weak Areas from memory), and Phase 4 (Rapid Review), tailored to the student's daily available hours.

Q3. Why did you use a Zero-LangChain architecture?

Answer: LangChain introduces heavy abstraction layers, slow startup overhead, frequent breaking API changes, and hard-to-debug stack traces. By writing clean, modular Python functions directly interfacing with Google's official SDK and FAISS, our app starts instantly, has full prompt transparency, and is completely maintainable.

🚀 4. How to Run the Project (Quick Reference)

Action	Command (PowerShell)
Navigate to Directory	`cd C:\Users\yusra\Downloads\intelliStudy\AI-Learning-Study-Assistant`
Run Streamlit App	`.\.venv\Scripts\python.exe -m streamlit run app.py`
Run Backend Test Suite	`.\.venv\Scripts\python.exe test_verification.py`
Run UI Flow Tests	`.\.venv\Scripts\python.exe test_app_flows.py`
Browser Access URL	`http://localhost:8501`